

ro.uvt.ri

Developer Onboarding & Workflow Guide

Team Q-UW9

April 25, 2026

Contents

1	Project Overview	2
1.1	Repository Structure	2
2	Prerequisites	2
3	Initial Setup (First Time Only)	2
3.1	Step 1 – Clone the Repository	2
3.2	Step 2 – Start WordPress (via wp-env + Docker)	3
3.3	Step 3 – Install Frontend Dependencies	3
3.4	Step 4 – Configure the API Connection	3
3.5	Step 5 – Start the React Development Server	3
4	Alternative: WordPress via Laragon / XAMPP	3
5	Daily Workflow	4
5.1	Starting a Work Session	4
5.2	Creating a Feature Branch	4
5.3	Keeping Up to Date	4
5.4	Ending a Work Session	5
6	Useful Commands Reference	5
7	Troubleshooting	5
7.1	“Permission Denied” on Windows	5
7.2	Port Conflicts	5
7.3	CORS Errors When Fetching from WordPress	5
8	Summary	6

1 Project Overview

This project follows a **Headless CMS** architecture:

- **Backend (CMS):** WordPress – used exclusively for content management via the REST API. Each developer runs a local instance.
- **Frontend:** React (Vite + TypeScript) – the actual website that end-users interact with. It consumes the WordPress REST API.

1.1 Repository Structure

```
ro.uvt.ro/  
|-- .gitignore          # Ignores WP core, node_modules, env files  
|-- .wp-env.json        # WordPress environment configuration  
|-- README.md           # Quick-start instructions  
|-- docs/               # This guide and other documentation  
|   +-- developer-guide.tex  
+-- frontend/           # React application (Vite + TypeScript)  
    |-- package.json  
    |-- src/  
    |   |-- App.tsx  
    |   |-- api/wordpress.js  
    |   +-- pages/  
    +-- vite.config.ts
```

Warning

The `wordpress/` folder on your local machine is **not tracked by Git**. Each developer sets up their own local WordPress instance using `wp-env` (Docker) or a local server stack such as Laragon or XAMPP.

2 Prerequisites

Before cloning the repository, make sure the following tools are installed:

1. **Git** – <https://git-scm.com/downloads>
2. **Node.js** (v18 or newer, LTS recommended) – <https://nodejs.org/>
3. **Docker Desktop** (required for `wp-env`) – <https://www.docker.com/products/docker-desktop/>

Tip

If you prefer not to use Docker, you can use **Laragon** (Windows) or **MAMP/XAMPP** instead. See Section 4 for the alternative WordPress setup.

3 Initial Setup (First Time Only)

3.1 Step 1 – Clone the Repository

```
git clone https://github.com/Q-UW9/ro.uvt.ro.git  
cd ro.uvt.ro
```

3.2 Step 2 – Start WordPress (via wp-env + Docker)

Make sure **Docker Desktop** is running, then execute:

```
npm -g install @wordpress/env  
wp-env start
```

This will:

- Download the WordPress core automatically.
- Create a local MySQL database inside Docker.
- Expose WordPress at `http://localhost:8888`.
- Provide an admin dashboard at `http://localhost:8888/wp-admin/` (default credentials: admin / password).

3.3 Step 3 – Install Frontend Dependencies

```
cd frontend  
npm install
```

3.4 Step 4 – Configure the API Connection

Create a `.env.local` file inside the `frontend/` directory:

```
# frontend/.env.local  
VITE_WP_API_URL=http://localhost:8888/wp-json
```

Warning

The `.env.local` file is **not tracked by Git** (it is listed in `.gitignore`). Each developer must create their own copy.

3.5 Step 5 – Start the React Development Server

```
npm run dev
```

The React application will start at `http://localhost:5173`.
You can now open both URLs side by side:

- **Add content:** `http://localhost:8888/wp-admin/`
- **View the website:** `http://localhost:5173`

4 Alternative: WordPress via Laragon / XAMPP

If Docker is not available, you can set up WordPress manually:

1. Download WordPress from <https://wordpress.org/download/>.
2. Extract it into your local server's web root (e.g., `C:\laragon\www\ro.uvt.ri`).
3. Create a MySQL database (e.g., `ro_uvt_ri`) using phpMyAdmin or HeidiSQL.

4. Copy `wp-config-sample.php` to `wp-config.php` and update the database credentials:

```
define( 'DB_NAME',      'ro_uvt_ri' );
define( 'DB_USER',      'root' );
define( 'DB_PASSWORD', '' );
define( 'DB_HOST',      'localhost' );
```

5. Open `http://ro.uvt.ri.test/wp-admin/` in your browser to complete the WordPress installation wizard.
6. Update `frontend/.env.local` to point to your local URL:

```
VITE_WP_API_URL=http://ro.uvt.ri.test/wp-json
```

5 Daily Workflow

5.1 Starting a Work Session

```
# 1. Pull the latest changes
git pull origin main

# 2. Start the WordPress backend (Docker method)
wp-env start

# 3. Start the React frontend
cd frontend
npm run dev
```

5.2 Creating a Feature Branch

Always work on a dedicated branch – never commit directly to `main`:

```
# Create and switch to a new branch
git checkout -b feature/your-feature-name

# ... make your changes ...

# Stage and commit
git add .
git commit -m "feat: describe your change"

# Push the branch to GitHub
git push origin feature/your-feature-name
```

Then open a **Pull Request** on GitHub to merge into `main`.

5.3 Keeping Up to Date

Before starting new work, always synchronize with the latest `main`:

```
# Switch to main and pull the latest
git checkout main
git pull origin main
```

```
# Switch back to your feature branch and rebase
git checkout feature/your-feature-name
git rebase main
```

Tip

If new dependencies were added by a teammate, you may need to re-run `npm install` inside the `frontend/` directory after pulling.

5.4 Ending a Work Session

```
# Stop the WordPress backend (Docker method)
wp-env stop
```

6 Useful Commands Reference

Command	Description
<code>wp-env start</code>	Start the local WordPress instance (Docker)
<code>wp-env stop</code>	Stop the local WordPress instance
<code>wp-env clean all</code>	Reset WordPress to a fresh install
<code>cd frontend && npm install</code>	Install frontend dependencies
<code>cd frontend && npm run dev</code>	Start React dev server (port 5173)
<code>cd frontend && npm run build</code>	Build production bundle
<code>cd frontend && npm run lint</code>	Run ESLint checks
<code>git pull origin main</code>	Sync with latest remote changes
<code>git rebase main</code>	Rebase current branch onto main
<code>git stash</code>	Temporarily shelve uncommitted work

7 Troubleshooting

7.1 “Permission Denied” on Windows

If Git reports dubious ownership errors:

```
git config --global --add safe.directory 'D:/path/to/ro.uvt.ri'
```

7.2 Port Conflicts

If port 8888 or 5173 is already in use:

- For WordPress: edit `.wp-env.json` and add a custom port mapping.
- For React: Vite will automatically try the next available port (5174, 5175, etc.).

7.3 CORS Errors When Fetching from WordPress

Install and activate the [WP GraphQL](#) or a CORS plugin on your local WordPress instance. Alternatively, configure the Vite proxy in `frontend/vite.config.ts`:

```
// vite.config.ts
export default defineConfig({
  server: {
    proxy: {
      '/wp-json': 'http://localhost:8888'
    }
  }
})
```

8 Summary

Clone → wp-env start → npm install → npm run dev → Start coding!

For questions or issues, open an issue on the GitHub repository:
<https://github.com/Q-UW9/ro.uvt.ri>